

Task 2 - DSA Algorithms

Prompt

Act as a Data Structures and Algorithms expert.

Generate implementations of the following algorithms in Python:

1) Binary Search

2) Merge Sort

3) Dijkstra's Algorithm

that are easy to understand .

For each algorithm, provide:

- Problem description

- Algorithm explanation

- Step-by-step working

- Python code with comments

- Sample input and output

- Time Complexity (Best, Average, Worst Case)

- Space Complexity

- Advantages and disadvantages

- Possible optimizations

Use clear and student-friendly language.

Output

1. Binary Search

Problem Description

Given a sorted array and a target value, find the position (index) of the target in the array.

Example:

```
Array: [2, 4, 6, 8, 10, 12]
Target: 8
Output: 3
```



Algorithm Explanation

Binary Search works by repeatedly dividing the search space into two halves.

1. Find the middle element.
2. If the middle element is the target, return its index.
3. If the target is smaller, search the left half.
4. If the target is larger, search the right half.
5. Repeat until the target is found or the search space becomes empty.

Step-by-Step Working

Array:

```
[2, 4, 6, 8, 10, 12]
```



Target = 10

Step 1

```
low = 0, high = 5
mid = (0 + 5) // 2 = 2
arr[mid] = 6
```



Since $10 > 6$, search right half.

Step 2

```
low = 3, high = 5
mid = (3 + 5) // 2 = 4
arr[mid] = 10
```



Target found at index 4.

Python Code with Comments

 Python

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        # Find middle index
        mid = (low + high) // 2

        # Target found
        if arr[mid] == target:
            return mid

        # Search left half
        elif target < arr[mid]:
            high = mid - 1

        # Search right half
        else:
            low = mid + 1

    # Target not found
    return -1

# Example usage
arr = [2, 4, 6, 8, 10, 12]
target = 10

result = binary_search(arr, target)
print("Index:", result)
```

Sample Input and Output

Input

Array = [2, 4, 6, 8, 10, 12]
Target = 10



Output

Index: 4



Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

Space Complexity



$O(1)$



Advantages

- Very fast for large sorted datasets.
- Requires little memory.
- Efficient searching.

Disadvantages

- Array must be sorted.
- Not suitable for linked lists.

Possible Optimizations

- Use Recursive Binary Search.
- Use interpolation search for uniformly distributed data.
- Use built-in modules like `bisect` in Python.

2. Merge Sort

Problem Description

Sort an unsorted array in ascending order using the Divide and Conquer approach.

Example:

Input: [38, 27, 43, 3, 9, 82, 10]
Output: [3, 9, 10, 27, 38, 43, 82]



Algorithm Explanation

Merge Sort follows three steps:

1. Divide the array into two halves.
2. Recursively sort each half.
3. Merge the sorted halves.

Step-by-Step Working

Array:

[38, 27, 43, 3]



Divide

[38, 27] [43, 3]



Divide Again

[38] [27] [43] [3]



Merge

[27, 38]
[3, 43]



Final Merge

[3, 27, 38, 43]



Python Code with Comments

Python



```
def merge_sort(arr):  
  
    # Base case  
    if len(arr) <= 1:  
        return arr  
  
    # Split array into two halves  
    mid = len(arr) // 2  
  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
  
    # Merge sorted halves  
    return merge(left, right)  
  
def merge(left, right):  
    result = []  
  
    i = 0  
    j = 0  
  
    # Compare elements from both arrays  
    while i < len(left) and j < len(right):  
  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
  
    # Add remaining elements  
    result.extend(left[i:])  
    result.extend(right[j:])  
  
    return result  
  
# Example usage  
arr = [38, 27, 43, 3, 9, 82, 10]  
  
sorted_arr = merge_sort(arr)
```



Sample Input and Output

Input

```
[38, 27, 43, 3, 9, 82, 18]
```



Output

```
Sorted Array: [3, 9, 18, 27, 38, 43, 82]
```



Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Space Complexity

```
 $O(n)$ 
```



Advantages

- Stable sorting algorithm.
- Guaranteed $O(n \log n)$ performance.
- Good for large datasets.

Disadvantages

- Requires extra memory.
- Slower than Quick Sort in some practical cases.

Possible Optimizations

- Use insertion sort for very small subarrays.
- Implement iterative (bottom-up) merge sort.
- Avoid unnecessary array copying.

3. Dijkstra's Algorithm

Problem Description

Find the shortest path from a source node to all other nodes in a weighted graph with non-negative edge weights.



Example:

```
A --4--> B
A --1--> C
C --2--> B
```



Shortest path from A to B is:

```
A → C → B = 3
```



Algorithm Explanation

Dijkstra's Algorithm works greedily:

1. Start from the source node.
2. Assign distance 0 to the source and infinity to others.
3. Pick the node with the smallest known distance.
4. Update distances of its neighbors.
5. Repeat until all nodes are processed.

A priority queue (min-heap) is used to efficiently select the closest node.

Step-by-Step Working

Graph:

```
A --4--> B
A --1--> C
C --2--> B
B --1--> D
C --5--> D
```



Initial Distances

```
A = 0
B = ∞
C = ∞
D = ∞
```



Visit A

```
B = 4
C = 1
```



Visit C

```
B = min(4, 1+2) = 3
D = min(∞, 1+5) = 6
```



Visit B

```
D = min(6, 3+1) = 4
```



Final Distances

```
A = 0
B = 3
C = 1
D = 4
```



Python Code with Comments

```
</> Python
```



```
import heapq

def dijkstra(graph, start):

    # Store shortest distances
    distances = {node: float('inf') for node in graph}
    distances[start] = 0

    # Priority queue
    pq = [(0, start)]

    while pq:

        current_distance, current_node = heapq.heappop(pq)

        # Skip outdated entries
        if current_distance > distances[current_node]:
            continue

        # Check neighbors
        for neighbor, weight in graph[current_node]:

            distance = current_distance + weight

            # Found a shorter path
            if distance < distances[neighbor]:
                distances[neighbor] = distance

                heapq.heappush(
                    pq,
                    (distance, neighbor)
                )

    return distances

# Example graph
graph = {
    'A': [('B', 4), ('C', 1)],
    'B': [('D', 1)],
    'C': [('B', 2), ('D', 5)],
    'D': []
}

result = dijkstra(graph, 'A')

print(result)
```


Sample Input and Output

Input Graph

```
A -> B (4)
A -> C (1)
C -> B (2)
B -> D (1)
C -> D (5)
```



Output

```
{
  'A': 0,
  'B': 3,
  'C': 1,
  'D': 4
}
```



Time Complexity

Using Min Heap (Priority Queue)

Case	Complexity
Best	$O((V + E) \log V)$
Average	$O((V + E) \log V)$
Worst	$O((V + E) \log V)$

Where:

- V = Number of vertices (nodes)
- E = Number of edges

Space Complexity

$O(V)$



(plus storage for the graph itself)

Advantages

- Finds shortest paths efficiently.
- Works well on sparse graphs.
- Widely used in GPS and routing systems.

Disadvantages

- Cannot handle negative edge weights.
- More complex than simple graph traversal algorithms.



Possible Optimizations

- Use a Fibonacci Heap (theoretical improvement).
- Stop early if only one destination node is needed.
- Use A* Search when a heuristic is available.
- Use adjacency lists for better memory efficiency.

Quick Comparison

Algorithm	Purpose	Best	Average	Worst	Space
Binary Search	Search in sorted array	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Merge Sort	Sorting	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Dijkstra	Shortest Path	$O((V+E)\log V)$	$O((V+E)\log V)$	$O((V+E)\log V)$	$O(V)$

These three algorithms are among the most important DSA topics and are frequently asked in coding interviews and programming courses.